

Franka Target-Conditioned Model

Remote GPU Training Handoff | Operator Runbook

PURPOSE: Train the 29D target-aware FlowEBRRT edge model using the full 700,000-row dataset. The machine has 512 GB RAM, so encoded-data caching is enabled for faster epochs.

DATA	700,000 conditioned rows / 32 edges per row
CONDITION	Current state + relative target + goal-mode flag (29D)
TRAINING	60 epochs, batch 128, CUDA AMP, cached encodings
RETURN	best_inference.pt plus the run metrics and configuration

1. Required inputs

- Git branch: **LatestRS** from github.com/himanshugupta1009/FlowMRMP.
- Confirm the branch contains the target-conditioned builder, auditor, tests, and updated training script before cloning.
- Copy the two HDF5 files below separately from GitHub. Do not use the smoke dataset.

FILE	SIZE	SHA-256
franka_edge_bundle_200k_pool128_k32_n350000_max50_fullvalid.h5	7.54 GB	7444be109233803a663897d9a1f9e7c0d b82568bf4f4b3e809d7faa77f8636d7
franka_edge_bundle_200k_pool128_k32_n350000_max50_targetcond_v1.h5	139.7 MB	f935ff3a793638b50af631fff06428c0 d496140dc8e2d9387960f23649368823

2. Clone and place the data

```
git clone --branch LatestRS --single-branch https://github.com/himanshugupta1009/FlowMRMP.git
cd FlowMRMP
```

Create `dataset/franka_eb_dataset/` and put both HDF5 files in it:

```
FlowMRMP/
dataset/franka_eb_dataset/
  franka_edge_bundle_200k_pool128_k32_n350000_max50_fullvalid.h5
  franka_edge_bundle_200k_pool128_k32_n350000_max50_targetcond_v1.h5
```

Keep the files together: the compact companion resolves the 7.54 GB source by filename and verifies its hash.

3. Create the Python environment

Use Python 3.11 if available. Run the commands for the machine's operating system.

Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-franka.txt
```

Windows PowerShell

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements-franka.txt
```

Confirm PyTorch sees the GPU before proceeding:

```
python -c "import torch; ok=torch.cuda.is_available(); print('CUDA:',ok); print('GPU:',torch.cuda.get_device_name(0) if ok else 'NONE')"
```

STOP if CUDA is False. Install a CUDA-enabled PyTorch build compatible with the installed NVIDIA driver, then repeat the check.

4. Verify code and data

```
python -m unittest Tests.test_franka_target_conditioning
python scripts/audit_franka_target_conditioned_edge_dataset.py
dataset/franka_eb_dataset/franka_edge_bundle_200k_pool128_k32_n350000_max50_targetcond_v1.h5
```

Required result: all 3 tests report **OK**, and the audit JSON prints "passed": true with no errors.

5. Start full training

This command is valid in Linux and Windows PowerShell:

```
python scripts/train_franka_edge_flow_matching.py --output-dir trained_models/franka_target_conditioned_v1 --description target_conditioned_v1 --device cuda --epochs 60 --batch-size 128 --target-progress-weight 0.25 --target-recall-radius 0.40 --cache-encoded-dataset
```

The cache is approximately 34 GB plus source arrays. With 512 GB RAM, leave caching enabled. The initial cache build can take time but later epochs avoid repeatedly encoding the full edge set.

If CUDA reports out-of-memory, stop cleanly and rerun with `--batch-size 64`, then 32 if necessary. RAM caching is unrelated to GPU VRAM and should remain enabled.

6. Monitor and resume

Keep the training terminal open. In a second terminal, monitor GPU load:

```
nvidia-smi -l 5
```

Each run gets a timestamped directory under `trained_models/franka_target_conditioned_v1/`. It contains console logs, per-epoch CSV metrics, plots, samples, provenance, and checkpoints.

WATCH	EXPECTED BEHAVIOR
val_loss	Generally declines; best checkpoint updates when it improves.
val_target_progress	Generally declines as a generated edge gets closer to target.
val_target_recall	Generally rises; it is a diagnostic, not the final planner score.
GPU utilization	Should be active during batches; cache building is mainly CPU/RAM work.

Resume after interruption

```
python scripts/train_franka_edge_flow_matching.py --resume  
trained_models/franka_target_conditioned_v1/RUN_NAME/checkpoints/last.pt --device cuda --cache-encoded-dataset
```

Replace RUN_NAME with the timestamped run directory. Do not change model architecture or loss arguments while resuming.

7. Completion and files to return

- Confirm `status.json` says the run completed and `best_inference.pt` exists.
- Return `checkpoints/best_inference.pt` - this is the model used for planner inference.
- Also return `checkpoints/best.pt`, `config.json`, `training_summary.json`, `metrics/losses.csv`, and `console.log`.
- Best handoff: compress and return the entire timestamped run directory.

Do not rename or discard the configuration and metrics. They are needed to reproduce the checkpoint and diagnose the 100-problem planner benchmark.

8. What this run does not prove

A lower validation loss alone does not prove FlowEBRRT is better than VanillaRRT. After receiving the checkpoint, the project owner must connect the planner's 29D inference condition and rerun the fixed 100-problem benchmark for success rate, successful-only mean/median planning time, and path execution time.